

Índice

1. Introducción y Diagnóstico Inicial del Entorno	2
2. Creación y Aprovisionamiento de Contenedores LXC en Proxmox VE	2
2.1. Especificaciones Técnicas de Recursos	2
2.2. Procedimiento Paso a Paso en la Interfaz de Proxmox VE	2
3. Fase 1: Compilación Standalone en Entorno Local (Windows)	3
3.1. Configuración de Next.js	3
3.2. Proceso de Automatización y Empaquetado Local	4
4. Fase 2: Transferencia de Archivos mediante Salto de Red Externo	4
5. Fase 3: Despliegue y Configuración del Frontend (LXC 143)	5
5.1. Instalación del Motor de Ejecución Node.js v22	5
5.2. Descarga de Archivos y Extracción en el Directorio de Producción	5
5.3. Inicialización de Variables de Entorno	5
5.4. Lanzamiento del Servidor en Segundo Plano	6
6. Fase 4: Configuración, Mitigación y Optimización de PostgreSQL (LXC 144)	6
6.1. Instalación del Motor de Base de Datos	6
6.2. Acceso a la Consola de PostgreSQL y Resolución del Error 500 de Codificación	6
6.3. Optimización de Recursos Críticos para 128 MB de RAM	7
6.4. Configuración de la Red Interna de Seguridad	8
7. Fase 5: Estrategia de Acceso y Despliegue en la Infraestructura Institucional	8
8. Conclusión del Estado de la Arquitectura	8
9. Glosario Técnico de Comandos de Terminal	10

1. Introducción y Diagnóstico Inicial del Entorno

El presente documento expone los criterios de diseño, contingencias técnicas y el procedimiento paso a paso ejecutado para el despliegue de una solución web monolítica desacoplada (Frontend en Next.js y persistencia relacional en PostgreSQL). El entorno de destino corresponde a la infraestructura de la NAP (Núcleo de Asistencia de Redes) de la facultad.

El despliegue representó un desafío crítico de administración de sistemas debido a tres condicionantes perimetrales estrictos:

- **Restricción Extrema de Hardware:** Un límite estricto e infranqueable de **128 MB de memoria RAM** utilizable por cada nodo de cómputo virtualizado.
- **Aislamiento de Red Perimetral:** Inexistencia de una red privada virtual (VPN) externa configurada en la máquina de desarrollo local. Esto impide el direccionamiento directo y el ruteo de paquetes hacia el segmento privado universitario (172.16.90.X).

2. Creación y Aprovisionamiento de Contenedores LXC en Proxmox VE

Para garantizar el aislamiento de procesos y optimizar el uso de los recursos del hipervisor, se optó por el aprovisionamiento de dos contenedores Linux (*LXC*) independientes corriendo sobre Debian 12.

2.1. Especificaciones Técnicas de Recursos

La distribución de hardware y direccionamiento de red se configuró según los parámetros consolidados en la Tabla 1.

Cuadro 1: Aprovisionamiento de Nodos LXC en Proxmox

Componente	Servidor Frontend (LXC 143)	Servidor Base de Datos (LXC 144)
Plantilla de OS	debian-12-standard_amd64.tar.zst	debian-12-standard_amd64.tar.zst
Cómputo (vCPU)	1 Core (Limitado a host)	1 Core (Limitado a host)
Memoria RAM	128 MB	128 MB
Memoria Swap	0 MB	0 MB
Almacenamiento	8 GB (Local-SSD ext4)	8 GB (Local-SSD ext4)
Interfaz de Red	eth0 (Bridge: vmbr0)	eth0 (Bridge: vmbr0)
Dirección IP IPv4	Static: 172.16.90.143/24	Static: 172.16.90.144/24
Puerta de Enlace	172.16.90.1	172.16.90.1
DNS Server	172.16.90.1	172.16.90.1

2.2. Procedimiento Paso a Paso en la Interfaz de Proxmox VE

El aprovisionamiento de los entornos se realizó siguiendo de forma estricta la siguiente secuencia técnica dentro del panel de administración web:

1. Se navegó hasta el almacenamiento local del nodo y se verificó la existencia de la plantilla oficial de Debian 12.
2. Se hizo clic en **Create CT** (Crear Contenedor) en la esquina superior derecha del panel.
3. **Pestaña General:** Se asignó el ID del contenedor (143), el nombre de host (45275660A) y se configuró una contraseña segura para el usuario administrador `root`.
4. **Pestaña Template:** Se seleccionó la plantilla de Debian del paso 1.
5. **Pestaña Root Disk:** Se fijó el tamaño del disco en 8 GB.
6. **Pestaña CPU:** Se asignó 1 Core.
7. **Pestaña Memory:** Se configuró la memoria RAM estrictamente en 128 MB y la Swap en 0 MB.
8. **Pestaña Network:** Se configuró la tarjeta `eth0`. Se seleccionó el modo **Static** en la configuración de IPv4, ingresando el CIDR correspondiente (172.16.90.143/24) y estableciendo la Gateway en 172.16.90.1.
9. Se repitió exactamente el mismo procedimiento para el segundo contenedor asignándole el ID 144, el hostname 45275660DB y la dirección IP estática 172.16.90.144/24.

3. Fase 1: Compilación Standalone en Entorno Local (Windows)

El proceso tradicional de construcción en Next.js requiere la ejecución de `npm run build`, el cual levanta subprocesos de compilación de código JavaScript/TypeScript sumamente pesados (con demandas transitorias de entre 500 MB y 1 GB de RAM). Intentar realizar este paso dentro del contenedor LXC de 128 MB causaría que el kernel de Linux abortara la tarea de inmediato mediante el *OOM Killer*.

3.1. Configuración de Next.js

Para mitigar esta barrera física, se configuró el proyecto de forma local para compilar bajo el esquema optimizado de producción independiente. Se añadió la siguiente directiva dentro del archivo estructural `next.config.js`:

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  typescript: {
    ignoreBuildErrors: true,
  },
  images: {
    unoptimized: true,
  },
  output: 'standalone',
  basePath: '/45275660',
}
```

```
export default nextConfig
```

Esta instrucción le ordena al motor compilar y trazar un mapa de dependencias selectivo, aislando únicamente los archivos binarios estrictamente obligatorios de producción dentro del subdirectorio `.next/standalone`, descartando herramental de desarrollo y dependencias redundantes.

3.2. Proceso de Automatización y Empaquetado Local

Dado que la compilación nativa en modo `standalone` de Next.js no arrastra automáticamente las carpetas de recursos estáticos (`public` y `.next/static`), se diseñó un script de comandos por lotes en el entorno de desarrollo local (Windows CMD). Este proceso automatiza la compilación, inyecta los directorios estáticos requeridos dentro de la carpeta independiente y consolida la compresión final en la raíz:

```
npm run build

xcopy /E /I /Y public .next\standalone\public
xcopy /E /I /Y .next\static .next\standalone\.next\static

cd .next\standalone
C:\TFI\FrontEnd\.next\standalone> tar -czf ../../blog_npm.tar.gz .
C:\TFI\FrontEnd\.next\standalone> cd ../../
```

Este flujo garantiza que el archivo binario resultante `blog_npm.tar.gz` se comporte como un servidor Node.js puramente autónomo, liviano y 100 % autocontenido, listo para su distribución.

4. Fase 2: Transferencia de Archivos mediante Salto de Red Externo

A causa de la falta de una VPN, la ejecución de transferencias directas vía protocolo seguro como `scp blog_npm.tar.gz root@172.16.90.143:/tmp` arrojaba sistemáticamente errores de tiempo de espera agotado (*Connection timed out*). No obstante, la arquitectura de red perimetral de la NAP de la facultad permitía conexiones de salida hacia la WAN (Internet) a través del puerto seguro de navegación HTTPS.

Para instrumentar la transferencia del archivo comprimido desde el entorno local de Windows hacia el contenedor, se aplicó la técnica de puenteo web usando la interfaz de **Filebin**:

1. En la computadora física de desarrollo (Windows), se abrió el navegador web y se navegó al sitio oficial de intercambio de datos para desarrolladores: <https://filebin.net/>.
2. Mediante la interfaz gráfica del sitio, se subió el archivo local `blog_npm.tar.gz` dentro de la zona de carga activa del navegador.
3. Se esperó a que la barra de carga finalizara la transferencia del binario real de varios megabytes.

4. Al completarse la subida, el nombre del archivo apareció listado en el portal. Se hizo **clic derecho** sobre el enlace de descarga del archivo y se seleccionó la opción **ⓈCopiar dirección de enlace** (*Copy link address*) para almacenar la URL directa y cruda en el portapapeles.

5. Fase 3: Despliegue y Configuración del Frontend (LXC 143)

5.1. Instalación del Motor de Ejecución Node.js v22

Se abrió la consola web de Proxmox correspondiente al contenedor frontend (**LXC 143**). Dado que los repositorios base de Debian suelen contar con versiones desactualizadas incompatibles con las API de Next.js moderno, se inyectó el repositorio oficial de Node-Source y se instaló el motor en su versión 22:

```
apt update && apt install -y curl
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt install -y nodejs
```

Se comprobó la correcta instalación validando el entorno mediante el comando `node -v`, el cual devolvió la confirmación del entorno de producción.

5.2. Descarga de Archivos y Extracción en el Directorio de Producción

Haciendo uso del link directo copiado previamente desde la interfaz de Filebin, se invocó la utilidad de descarga `wget` aplicando el flag de mitigación de errores de certificados por proxy estudiantil (`-no-check-certificate`). Posteriormente se limpiaron residuos de pruebas fallidas anteriores y se procedió a desempaquetar la estructura:

```
wget --no-check-certificate "URL_DE_FILEBIN" -O /tmp/blog_npm.tar.gz

mkdir -p /opt/blog
cd /opt/blog

tar -xzf /tmp/blog_npm.tar.gz
rm /tmp/blog_npm.tar.gz
```

5.3. Inicialización de Variables de Entorno

Para que las llamadas asincrónicas a la API de Next.js puedan comunicarse de forma interna con el clúster de la base de datos sin salir a internet, se procedió a crear y editar el archivo de configuración de variables mediante el editor nativo de terminal `nano`:

```
nano .env
```

Dentro del editor, se redactaron minuciosamente los parámetros de interconexión del segmento privado apuntando a la dirección IP estática del nodo adyacente de base de datos (172.16.90.144):

```
DB_HOST=172.16.90.144
DB_PORT=5432
DB_NAME=blogdb
DB_USER=postgres
DB_PASSWORD=45275660
PORT=3000
NEXT_PUBLIC_BASE_PATH=/45275660
```

Se guardaron las modificaciones presionando la combinación de teclas **Ctrl + O**, se confirmó con **Enter** y se cerró la interfaz del editor mediante **Ctrl + X**.

5.4. Lanzamiento del Servidor en Segundo Plano

Con el fin de liberar la consola de comandos del contenedor para tareas posteriores y mantener el servicio corriendo de forma persistente, se arrancó la aplicación Next.js standalone forzando las directivas de producción y usando el operador de bifurcación (**&**):

```
NODE_ENV=production node --env-file=.env server.js &
```

6. Fase 4: Configuración, Mitigación y Optimización de PostgreSQL (LXC 144)

6.1. Instalación del Motor de Base de Datos

Se ingresó a la pestaña de consola web del contenedor de persistencia (**LXC 144**) en Proxmox y se instaló el clúster oficial de PostgreSQL junto con sus módulos de extensiones:

```
apt update && apt install -y postgresql postgresql-contrib
```

6.2. Acceso a la Consola de PostgreSQL y Resolución del Error 500 de Codificación

Al realizar las pruebas de navegación iniciales, el Frontend de Next.js retornaba códigos de error de Servidor Interno (HTTP 500). Mediante el análisis exhaustivo de registros redirigidos, se desveló el siguiente mensaje crítico arrojado por el controlador: `invalid byte sequence for encoding UTF8": 0xc2 0x48`.

Este error surge porque la plantilla limpia de LXC inicializa por defecto el servidor de base de datos bajo el esquema regional restrictivo `SQL_ASCII`. Dado que el código del frontend transmite objetos de consulta codificados estrictamente bajo el estándar UTF-8, el motor abortaba la ejecución de los hilos de consulta.

Para subsanar el inconveniente, se requería acceder al núcleo interactivo del motor, eliminar la base errónea y forzar la creación de una nueva estructura utilizando la plantilla de sistema puro `template0` para eludir las restricciones regionales de herencia:

1. En la shell del LXC 144, se escalaron privilegios para adoptar la identidad del usuario administrador nativo del motor de bases de datos:

```
root@45275660A:~# su - postgres
```

2. Se ingresó de manera directa a la terminal interactiva del sistema de base de datos invocando su binario:

```
postgres@45275660A:~$ psql
```

3. Una vez desplegado el prompt interactivo `postgres=#`, se ejecutaron las siguientes sentencias estructuradas de reparación y aprovisionamiento:

```
-- 1. Crear la base de datos en UTF8 utilizando template0
CREATE DATABASE blogdb WITH ENCODING = 'UTF8' TEMPLATE = template0;

-- 2. Actualizar la contraseña del usuario administrador
ALTER USER postgres WITH PASSWORD '45275660';

-- 3. Conectarse a la nueva base
\c blogdb

-- 4. Crear la tabla requerida por la API
CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 5. Insertar un registro de prueba
INSERT INTO posts (title, content)
VALUES ('Primer post en la NAP', 'Hola mundo');

-- 6. Salir de PostgreSQL
\q

-- 7. Salir del usuario postgres
exit
```

6.3. Optimización de Recursos Críticos para 128 MB de RAM

Dado que PostgreSQL está configurado por defecto para servidores con altos recursos, dejar los parámetros de fábrica provocaría un desborde inmediato de memoria en el contenedor LXC, causando la baja del servicio. Se procedió a editar los archivos de configuración del clúster mediante privilegios de `root`:

```
nano /etc/postgresql/15/main/postgresql.conf
```

Se localizaron las siguientes directivas y se restringió su consumo para ajustarlo al entorno embebido:

<code>listen_addresses = '*'</code>	# Habilitar la escucha en todas las interfaces
<code>shared_buffers = 10MB</code>	# Reducir drásticamente la memoria asignada a caché
<code>work_mem = 1MB</code>	# Minimizar la RAM

```
maintenance_work_mem = 8MB          # Ajustar el límite de operaciones de mantenimiento
```

6.4. Configuración de la Red Interna de Seguridad

Para impedir el escaneo de puertos y asegurar que únicamente el contenedor del frontend pueda interactuar con los datos, se editó el cortafuegos interno de acceso de PostgreSQL:

```
nano /etc/postgresql/15/main/pg_hba.conf
```

Al final de dicho archivo, se eliminaron accesos genéricos y se configuró una regla de máscara IP estricta de host único (/32) apuntando de forma única al LXC 143:

#	TYPE	DATABASE	USER	ADDRESS	METHOD
host		blogdb	postgres	172.16.90.143/32	scram-sha-256

Se guardaron los cambios y se reinició el demonio del sistema operativo para instanciar la reconfiguración física de memoria y red:

```
systemctl restart postgresql@15-main
```

7. Fase 5: Estrategia de Acceso y Despliegue en la Infraestructura Institucional

Durante las etapas de desarrollo y pruebas de conectividad externa, se implementó un túnel SSH reverso temporal utilizando el servicio de Pinggy (a.pinggy.io) sobre el puerto seguro 443. Esto permitió eludir las restricciones perimetrales del firewall de la red interna y auditar el comportamiento del Frontend desde redes externas de forma temprana.

Sin embargo, para la puesta en producción definitiva y la correspondiente evaluación de la cátedra, se prescindió del uso de herramientas de tunelización externas. En su lugar, se procedió a integrar el contenedor LXC directamente con la infraestructura de red de la Facultad (la NAP).

Para ello, se coordinó con la administración del entorno el mapeo de la IP estática interna del Frontend (172.16.90.143) y su puerto de escucha (3000) dentro del proxy inverso institucional. Como resultado, la aplicación quedó publicada de forma estable y segura bajo el dominio oficial en la siguiente URL asignada:

<https://nap.frt.utn.edu.ar/45275660>.

Al ingresar a dicha dirección desde el navegador comercial de la máquina de Windows del hogar, el tráfico se canaliza de manera transparente y segura hacia el puerto 3000 del contenedor LXC interno de la universidad, logrando renderizar el Frontend e interactuar con los registros de base de datos de forma remota.

8. Conclusión del Estado de la Arquitectura

El despliegue ha sido completado con éxito, alcanzando un estado operativo óptimo sustentado bajo los principios de la ingeniería de sistemas restringidos.

Gracias a la segmentación y optimizaciones aplicadas, los consumos de memoria en producción se estabilizaron bajo métricas seguras:

- **Nodo Frontend (LXC 143):** Estabilizado en un consumo plano de apenas **70 MB de memoria RAM**, derivado de haber evitado el proceso de empaquetado interno de Node.js mediante el artefacto standalone previamente compilado.
- **Nodo de Persistencia (LXC 144):** Estabilizado en **60 MB de memoria RAM**, consecuencia de la reducción de los buffers compartidos del motor PostgreSQL.

Ambos servicios coexisten de manera holgada por debajo del límite estricto de 128 MB asignado por la cátedra, garantizando una excelente performance del ecosistema para la presentación y evaluación final del proyecto.

9. Glosario Técnico de Comandos de Terminal

Cuadro 2: Glosario de Comandos de Administración de Sistemas

Comando Técnico	Impacto y Acción Operativa en el Sistema
<code>apt update</code>	Sincroniza el mapa local de paquetes del sistema operativo contrastándolo con los servidores de distribución de Debian.
<code>apt install -y [paquete]</code>	Descarga e instala binarios del sistema (<code>nodejs</code> , <code>curl</code> , etc.) aprobando por defecto los flags de confirmación.
<code>xcopy /E /I /Y [origen] [destino]</code>	Copia archivos y árboles de directorios completos de Windows de forma recursiva, forzando el reemplazo y omitiendo advertencias.
<code>tar -czf [archivo.tar.gz] [rutas]</code>	Agrupar un árbol de carpetas del proyecto y aplica un algoritmo de compresión gzip sobre el bloque binario resultante.
<code>tar -xzf [archivo.tar.gz]</code>	Realiza la extracción y descompresión de ficheros empaquetados bajo la extensión de distribución comprimida <code>.tar.gz</code> .
<code>wget -no-check-certificate</code>	Inicia la descarga de un recurso remoto web instruyendo al cliente a omitir errores o alertas de cadenas de confianza SSL.
<code>rm -rf [ruta]</code>	Elimina un archivo o estructura de directorios de manera recursiva e incondicional, forzando la remoción del sistema.
<code>mkdir -p [ruta]</code>	Genera un directorio asegurando la creación de las carpetas padre anidadas sin arrojar fallos en caso de existir previamente.
<code>pkill -9 node</code>	Envía de forma masiva e irreversible la señal de interrupción forzada SIGKILL a todas las instancias activas de Node.js en memoria.
<code>node -env-file=.env server.js &</code>	Ejecuta el backend de Next.js inyectando variables de entorno físicas y ordenando al hilo correr en segundo plano.
<code>su - postgres</code>	Sustituye el entorno de usuario del shell actual para adoptar la identidad y los privilegios del administrador de PostgreSQL.
<code>psql</code>	Invoca la terminal interactiva de comandos de PostgreSQL para entablar consultas y transacciones sobre bases de datos relacionales.
<code>\c [nombre_base]</code>	Instrucción interna del prompt de <code>psql</code> para conmutar la sesión activa hacia el contexto de una base de datos específica.
<code>\q</code>	Finaliza y abandona la sesión interactiva del cliente de comandos de <code>psql</code> , restituyendo el control a la terminal de Linux.
<code>systemctl restart [servicio]</code>	Ordena al gestor de arranque Systemd reiniciar un demonio de fondo para forzar la lectura de nuevos archivos de configuración.
<code>ssh -p 443 -R [puertos] a.pinggy.io</code>	Instancia una sesión SSH reversa temporal con Pinggy utilizada en la fase de diagnóstico y pruebas de ruteo externo, previo al despliegue final en el proxy de la NAP.
<code>cd [ruta]</code>	Cambia el directorio de trabajo actual de la terminal hacia la ruta especificada del sistema de archivos.
<code>nano [archivo]</code>	Abre el editor de texto interactivo Nano para modificar archivos de configuración directamente desde la terminal Linux.
<code>curl -fsSL [URL]</code>	Realiza una solicitud HTTP silenciosa y descarga scripts o contenido remoto desde una dirección web.
<code>npm run build</code>	Ejecuta el script de compilación definido en el proyecto Next.js para generar el artefacto optimizado de producción.